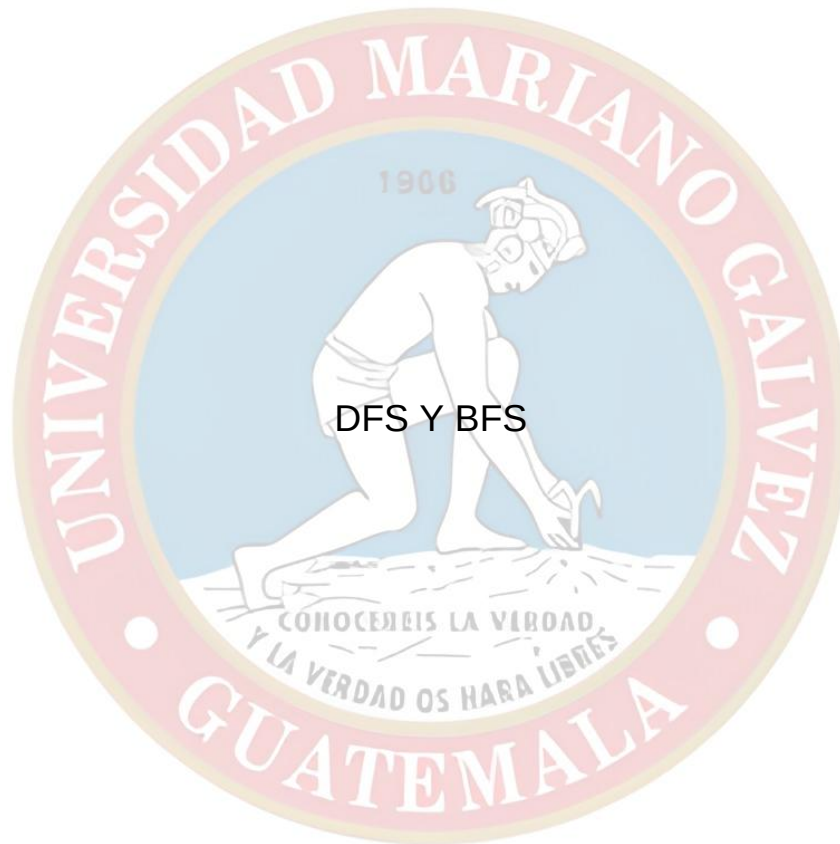




UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA
INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA
COMPUTACIÓN



ERICKA DANIELA CALDERA GÓMEZ / 9941-24-3190
BRANDON JOSÉ MARIO PRETZANTZIN PRETZANTZIN / 9941-24-20678
JOSÉ DANIEL VÉLIZ VELÁSQUEZ / 9941-24-12576

Guatemala, mayo 2026



Contenido

Parte I – Recorridos manuales de grafos	4
Parte 2– Comparación técnica C++ vs Java	9



Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Nombre:	Erika Daniela Caldera Gomez	Carné:	9941-24-8190
Nombre:	José Daniel Veliz Velasquez	Carné:	9941-24-12576
Nombre:	Brandon José Mario Prentzantzin Pretzantzin	Carné:	

Carrera	Ingeniería en Sistemas	Carrera	9941
Asignatura	Programación III	Curso	022
Ciclo	Quinto Ciclo	Jornada	Domingos
Catedrático	Ing. MBA David Alvarez	Fecha	24/05/2026
Semestre	Primer Semestre	Sección	B

Objetivo

Aplicar recorridos de grafos mediante BFS y DFS, representando gráficamente los grafos, resolviendo recorridos manualmente y comparando una implementación en C++ puro contra una implementación en Java con Collections Framework.

Parte 1 – Recorridos manuales de grafos

Instrucciones

Para cada ejercicio, el grupo debe:

1. Dibujar el grafo.
2. Escribir la lista de adyacencia.
3. Realizar el recorrido BFS desde el nodo indicado.
4. Realizar el recorrido DFS desde el nodo indicado.
5. Indicar el orden de visita.

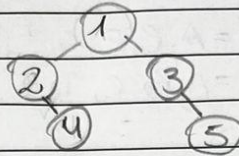
Cuando exista más de un vecino disponible, deben visitar los nodos en orden ascendente o alfabético.

Ejercicio 1 1-2 1-3 2-4 3-5 Inicio: 1	Ejercicio 2 1-2 1-4 2-3 4-5 5-6 Inicio: 1	Ejercicio 3 A-B A-C B-D B-E C-F Inicio: A
Ejercicio 4 1-2 2-3 3-4 4-5 Inicio: 1	Ejercicio 5 1-2 1-3 1-4 2-5 4-6 Inicio: 1	Ejercicio 6 A-B B-C C-D A-E E-F Inicio: A
Ejercicio 7 1-2 1-3 2-4 2-5 3-6 3-7 Inicio: 1	Ejercicio 8 A-B A-C B-D C-D D-E Inicio: A	Ejercicio 9 1-2 2-3 3-1 3-4 4-5 Inicio: 1
Ejercicio 10 A-B A-D B-C D-E C-F E-F Inicio: A	Ejercicio 11 1-2 1-5 2-3 2-4 5-6 6-7 Inicio: 1	Ejercicio 12 A-B A-C B-E C-D D-E E-F Inicio: A
Ejercicio 13 1-2 1-3 2-4 4-6 3-5 5-6 6-7 Inicio: 1	Ejercicio 14 A-B B-C C-D D-A A-E E-F Inicio: A	Ejercicio 15 1-2 1-3 2-5 3-4 4-6 5-6 6-7 7-8 Inicio: 1
Ejercicio 16 1-2 1-3 2-4 2-5 3-6	Dibuje el grafo. Escriba la lista de adyacencia. Realice el recorrido BFS desde el nodo 1. Realice el recorrido DFS desde el nodo 1. Responda: ¿cuándo BFS llega al nodo 9?	



Parte I – Recorridos manuales de grafos

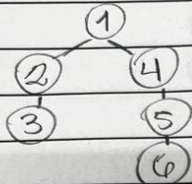
1.



BFS = 1, 2, 3, 4, 5

DFS = 1, 2, 4, 3, 5

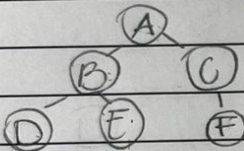
2.



BFS = 1, 2, 4, 3, 5, 6

DFS = 1, 2, 3, 4, 5, 6

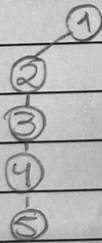
3.



BFS = A, B, C, D, E, F

DFS = A, B, D, E, C, F

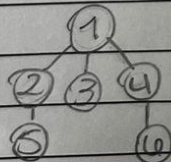
4.



BFS = 1, 2, 3, 4, 5

DFS = 1, 2, 3, 4, 5

5.

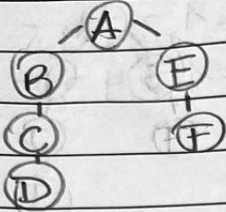


BFS = 1, 2, 3, 4, 5, 6

DFS = 1, 2, 5, 3, 4, 6

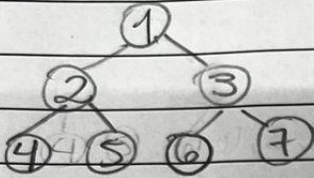


6.



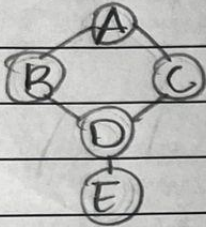
BFS = A, B, E, C, F, D
DFS = A, B, C, D, E, F

7.



BFS = 1, 2, 3, 4, 5, 6, 7
DFS = 1, 2, 4, 5, 3, 6, 7

8.



BFS = A, B, C, D, E
DFS = A, B, D, E, C



Q.

BFS = 1, 2, 3, 4, 5
DFS = 1, 2, 3, 4, 5

1.

BFS = A, B, D, C, E, F
DFS = A, B, C, F, D, E

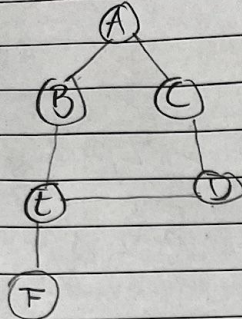
II.

BFS = 1, 2, 3, 4, 5, 6, 7
DFS = 1, 2, 3, 4, 5, 6, 7

MY WAY



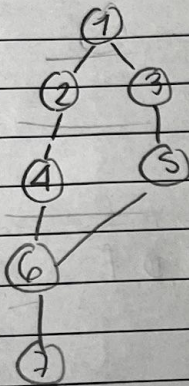
12.



BFS = A, B, C, E, D, F

DFS = A, B, E, F, C, D

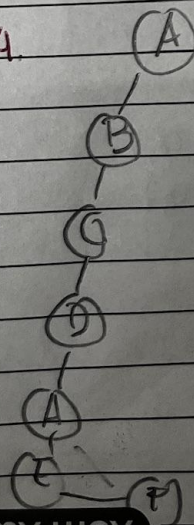
13.



BFS = 1, 2, 3, 4, 5, 6, 7

DFS = 1, 2, 4, 6, 7, 3, 5

14.



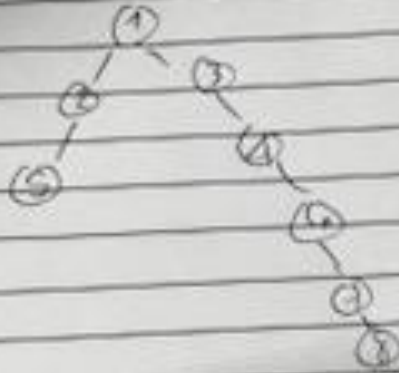
BFS = A, B, C, D, A, E, F

DFS = A, B, C, D, A, E, F

MY WAY



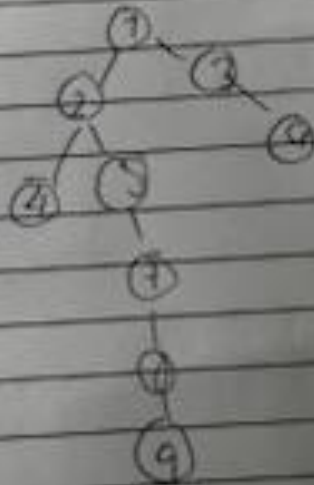
15.



BFS: 1, 2, 3

DFS: 1, 2, 5, 4, 6, 7, 8

16.



BFS: 1, 2, 3, 4, 5, 6, 7, 8, 9

DFS: 1, 2, 4, 5, 7, 8, 9, 6, 3



Parte 2– Comparación técnica C++ vs Java

The screenshot displays the Visual Studio Code interface with a C++ project. The Explorer panel on the left shows the file structure, including folders for 'PROGRAMACIÓN', 'Actividad_Grupo_DFS_BFS', and 'INCREDIBLE WATCHER'. The main editor shows the code for 'BFS_DFS.cpp', which defines structures for 'NodoAdyacente', 'Vertice', and 'NodoCola', and a 'Cola' class. The terminal at the bottom shows the execution of the program, outputting the results of BFS and DFS traversals. A message at the bottom indicates 'Compilation successful.'

```
1 #include <iostream>
2 using namespace std;
3
4 struct NodoAdyacente {
5     int destino;
6     NodoAdyacente* siguiente;
7 };
8
9 struct Vertice {
10     int id;
11     NodoAdyacente* lista;
12 };
13
14 struct NodoCola {
15     int dato;
16     NodoCola* siguiente;
17 };
18
19 class Cola {
20 private:
21     NodoCola* frente;
22     NodoCola* final;
23 public:
24     Cola() {
25         frente = nullptr;
26         final = nullptr;
27     }
28
29     bool estaVacia() {
30         return frente == nullptr;
31     }
32 }
```

TERMINAL

```
PS C:\Programación> cd 'c:\Programación\Programación III\Actividad_Grupo_DFS_BFS\output'
PS C:\Programación\Programación III\Actividad_Grupo_DFS_BFS\output> & .\BFS_DFS.exe
BFS: 1 3 2 4 5 6 7
DFS: 1 3 4 6 7 5 2
PS C:\Programación\Programación III\Actividad_Grupo_DFS_BFS\output>
```

0 errors
0 warnings

Compilation successful.

Salida en C++

Programación III
Sección B



```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window BFS_...
<default config>
Projects Services Files
BFS_DFS
  Source Packages
    com.mycompany.bfsDFS
      BFS_DFS.java
  Test Packages
  Dependencies
  Java Dependencies
  Project Files
  CLASE_5_OCT
    Source Packages
      com.mycompany.clase_5_oct
        CLASE_5_OCT.java
        Clase.java
  Test Packages
  Dependencies
  Java Dependencies
  Project Files
  CLASE_AGOSTO_31
bfs - Navigator
Members
BFS_DFS
  BFS_DFS()
  agregarArista(int origen, int destino)
  bfs(int inicio)
BFS_DFS.java
Source History
28 Set<Integer> visita
29 Queue<Integer> cola
30
31 visitado.add(inicio)
32 cola.add(inicio);
33
34 System.out.print("B
35
36 while (!cola.isEmpty()
37     int actual = co
38     System.out.prin
39
40     for (int vecino
41         if (!visita
42             visitad
43             cola.ad
44         }
45     }
46 }
47
48 System.out.println(
49 )
50
51 public void dfs(int ini
Output - Run (BFS_DFS)
--- compiler:3.13.0:compile (default-compile) @ BFS_DFS ---
Nothing to compile - all classes are up to date.
--- exec:3.1.0:exec (default-cli) @ BFS_DFS ---
BFS: 1 2 3 5 4 6 7
DFS: 1 2 5 6 4 3 7
-----
BUILD SUCCESS
-----
Total time: 2.051 s
Output 38:9 INS
```

Salida en Java



1. ¿Qué estructuras se implementaron manualmente en C++?

En C++ tocó crear varias cosas desde cero, como la lista de adyacencia, la cola y también manejar la memoria usando new y delete.

2. ¿Qué estructuras ya existen en Java?

Java ya trae muchas estructuras listas para usar, por ejemplo, HashMap, ArrayList, Queue, LinkedList y HashSet.

3. ¿Cuál código es más largo?

El código en C++ quedó más largo porque hubo que construir más cosas manualmente.

4. ¿Cuál código es más fácil de entender?

El de Java se entiende más rápido porque usa librerías ya hechas y el código queda más limpio.

5. ¿Qué lenguaje exige mayor control del programador?

C++ pide más control porque uno debe manejar memoria, punteros y estructuras manualmente.

6. ¿Qué ventajas tiene C++ en este caso?

C++ puede ser más rápido y eficiente, además ayuda a entender cómo funcionan realmente las estructuras internas.

7. ¿Qué ventajas tiene Java en este caso?

Java facilita mucho el trabajo porque trae herramientas listas y evita muchos errores de memoria.

8. ¿Qué sucede si en C++ no se libera memoria correctamente?

Puede haber fugas de memoria y el programa empieza a consumir más RAM de la necesaria.

9. ¿Por qué BFS necesita una cola?

Porque BFS trabaja por niveles y la cola mantiene el orden correcto de recorrido.

10. ¿Por qué DFS puede resolverse con recursión?

Porque DFS va profundizando nodo por nodo y la recursión ayuda a guardar automáticamente el camino recorrido.